

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Compliance Intelligence Platform

A Unified NLP-Based System for Startup State-Policy Fit and Contract Compliance Analysis.

Team Members:

SAI PRAVALI MOTAMARRI- 23MIC0086

VARANASI SREEJA-23MID0005

G. VARSHA -23MID0046

COURSE CODE: CSI4001

FACULTY NAME: DR. SHARMILA BANU K

Abstract

The Compliance Intelligence Platform is a web-based decision-support application designed to help startups and organisations review two related areas of compliance. First, it compares a startup's profile with selected state startup-policy requirements in Tamil Nadu and Karnataka. Second, it reviews contract clauses against a set of internal company policies and a curated set of state-law or regulatory candidates. The application brings structured policy data, document text extraction, clause-level analysis, semantic retrieval and rule-based comparisons into one Streamlit interface.

The platform has three implemented user-facing modules: (1) State Fit Advisor, (2) Contract vs Company Policies, and (3) Contract vs State Regulations. It uses a mixture of structured rules and NLP techniques. Semantic similarity is used to retrieve potentially relevant policy clauses; a separate structured reasoning layer checks quantities, permissions, prohibitions and other conditions to decide whether a conflict is supported. The system reports evidence and uncertainty rather than treating every similarity match as a legal conflict.

The project is a prototype and decision-support tool, not a substitute for legal advice. Some internal policies and test cases are synthetic. The state-law module uses sourced candidate records for review; its output does not by itself establish that every record is legally applicable or enforceable.

1. Title

Compliance Intelligence Platform: A Unified NLP-Based System for Startup State-Policy Fit and Contract Compliance Analysis.

2. Motivation

Startups and growing organisations operate within several layers of requirements. A startup may need to understand whether it meets the eligibility conditions for state incentives, what registrations or evidence it still needs, and which benefits may be available. At the same time, organisations enter into vendor, service and other commercial agreements. Contract terms may differ from the organisation's internal policies or from relevant statutory requirements.

These checks are often performed manually by reading long policy documents, company profiles and contracts. Manual review takes time, can overlook important clauses, and makes it difficult to compare the same requirement consistently across multiple documents.

The project was motivated by the need for a single, understandable tool that can organise these checks, identify potentially relevant clauses, compare structured conditions and show the evidence behind each result. The goal is to support early review and prioritise matters for human verification, not to make autonomous legal decisions.

3. Existing Gap / Problem That Motivated the Project

3.1 Startup policy discovery gap

- State startup policies contain eligibility rules, incentives, reimbursements, programme provisions and procedural steps spread across lengthy documents.
- A startup must compare its own facts, such as age, turnover, employee count, industry, location and recognition status, with multiple policy conditions.
- A missing fact should not automatically be treated as failure. Many manual comparisons do not clearly separate confirmed eligibility from missing evidence or competitive benefits.

3.2 Contract review gap

- A contract may contain many clauses, while the corresponding internal policy may be stored separately.
- A reviewer must identify clauses about the same topic and then compare actual terms, such as 15 days versus 30 days, or whether prior written consent is required.
- Text similarity alone is not enough: two clauses may discuss the same topic and still be compatible. Conversely, a numeric or permission mismatch may create a meaningful conflict.

3.3 Jurisdiction and evidence gap

- Contract terms may need to be checked against requirements associated with a particular state or national law.
- A useful result should preserve the source text, jurisdiction, page or section reference and source URL so that a reviewer can verify the underlying material.

The platform addresses these gaps by combining profile-based rule evaluation, document extraction, semantic candidate retrieval and deterministic conflict checks. The repository documentation also records known limitations, such as synthetic test policies, incomplete source coverage and the need for additional legal review.

4. Architecture

The application follows a layered architecture, shown in Figure 1. A Streamlit presentation layer collects inputs and displays results; three module-level workflows coordinate each use case; a shared core layer performs document processing, NLP retrieval and rule-based reasoning; and a data layer holds the structured policy, regulation and corpus files. The modules share common schemas and processing utilities where appropriate.

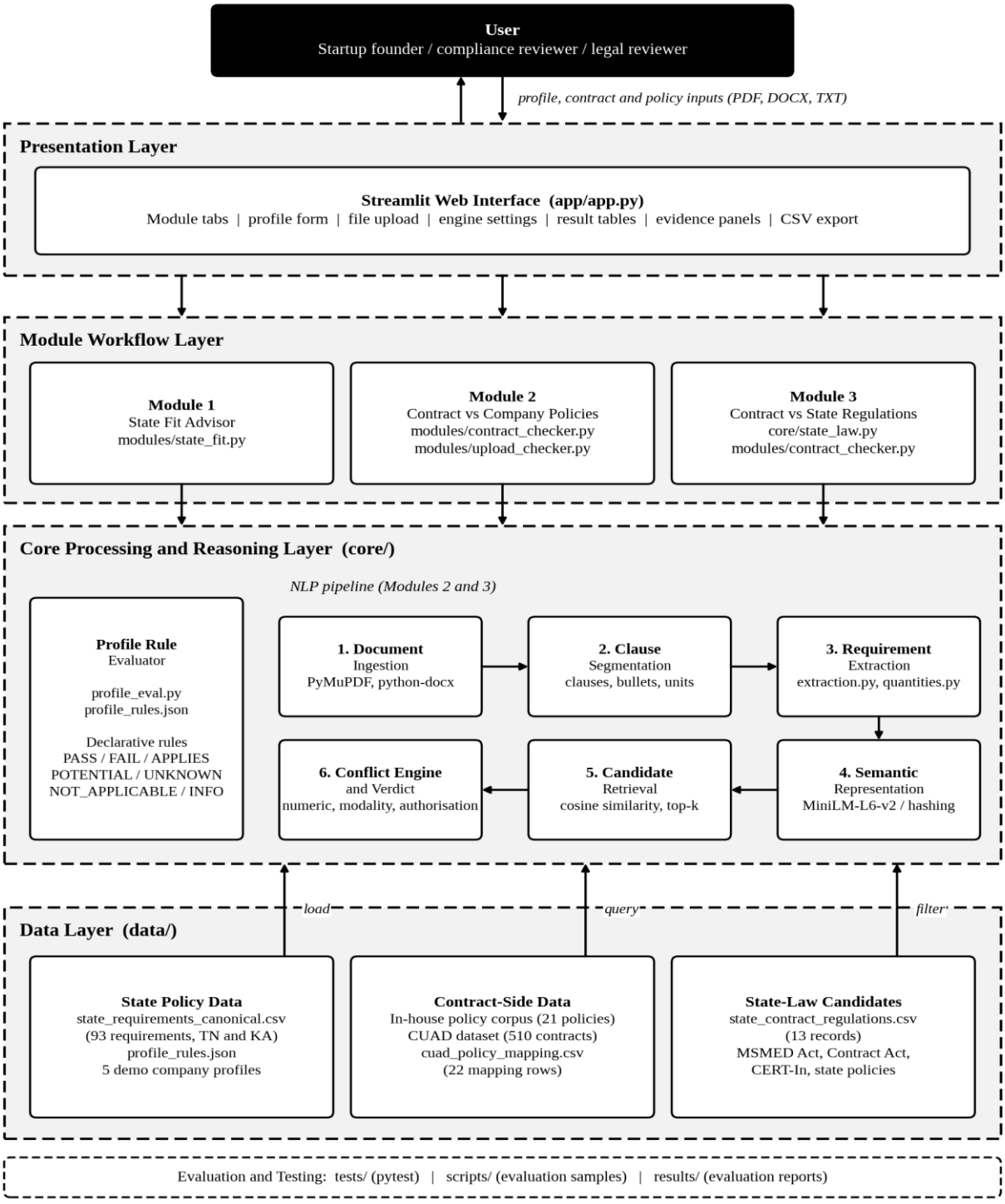


Figure 1: Layered architecture of the Compliance Intelligence Platform

The responsibilities of each layer are as follows.

- **Presentation layer:** app/app.py provides the Streamlit interface, module tabs, input forms, file upload, engine settings, result tables, evidence panels and CSV export.
- **Module workflow layer:** modules/state_fit.py handles startup state fit; modules/contract_checker.py and modules/upload_checker.py handle contract-versus-policy checking; core/state_law.py supplies state-law candidates for the statutory audit.
- **Core processing and reasoning layer:** core/engine.py, extraction.py, matching.py, embeddings.py, conflict.py, quantities.py and profile_eval.py implement document ingestion, requirement extraction, similarity retrieval, quantity comparison, profile rule evaluation and explanations.
- **Data layer:** data/state_policies/, data/policy_corpus/, data/profiles/ and data/cuad/ store state requirements, internal policies, demo profiles, mappings and benchmark data.
- **Evaluation and testing:** tests/, scripts/ and results/ contain automated tests, evaluation samples and evaluation reports.

4.1 High-level architecture flow

1. The user opens the Streamlit application and selects one of the three implemented modules.
2. Module 1 accepts a company profile and loads the structured state-policy dataset and declarative evaluation rules.
3. Module 2 accepts a contract and one or more internal policy documents (or uses the built-in policy corpus), extracts text and clauses, retrieves relevant policy lines, and applies structured conflict rules.
4. Module 3 accepts a target jurisdiction and contract text or clause, filters the state-law candidate dataset, and presents relevant sourced records for comparison.
5. Results are displayed with verdicts, explanations, evidence text and available source metadata. The user can review the output and verify the original source.

4.2 Module 1: State Fit Advisor

Purpose: To help a startup understand how its known profile facts compare with selected startup-policy requirements in Tamil Nadu and Karnataka.

- Inputs include company name, industry, legal form, startup age, annual turnover, employee count and other profile facts used by individual rules.
- The system loads 93 canonical state requirements across the two states and applies declarative rules stored in profile_rules.json.
- Each requirement receives a verdict such as PASS, FAIL, APPLIES, POTENTIAL, UNKNOWN, NOT_APPLICABLE or INFO.
- The system keeps missing information separate from failure. For example, if the profile does not state whether a company holds a particular certification, the result can be UNKNOWN rather than FAIL.
- The summary shows eligibility failures, actions to take, confirmed or potential benefits and facts that are still needed. The project deliberately does not claim that one state is universally best.

4.3 Module 2: Contract vs Company Policies

Purpose: To identify potential differences between a contract and the organisation's internal policies.

1. The user loads a demonstration scenario or uploads a contract and one or more company policy documents. Supported upload formats are PDF, DOCX and TXT.
2. Document text is extracted. PDF extraction uses PyMuPDF; DOCX extraction uses python-docx; TXT files are read as plain text.
3. The contract is divided into candidate clauses or text segments. Policy documents are divided into numbered requirements, bullet points or substantive text units, depending on their structure.
4. A requirement extractor identifies structured details such as obligation or permission, action, object, recipient, authorisation, quantities, units, direction, deadlines, conditions and exceptions.
5. Semantic retrieval ranks potentially relevant policy requirements. The default sentence-transformer model configured in the matcher is all-MiniLM-L6-v2; a deterministic hashing encoder is also available as an offline baseline.
6. A separate conflict engine compares structured information. It checks supported authorisation, numeric and modality conditions. Similarity is used to find relevant text, not to decide the conflict.

7. The result uses labels such as CONFLICT, PARTIAL_CONFLICT, NO_CONFLICT and UNCERTAIN, with evidence and explanations.

4.4 Module 3: Contract vs State Regulations

Purpose: To help a reviewer inspect contract terms against sourced state-specific statutory or regulatory candidate records.

- The user selects Tamil Nadu or Karnataka and supplies a clause or contract for review.
- The state-law loader filters records by jurisdiction and candidate requirement type (OBLIGATION, RESTRICTION or EXEMPTION). Startup benefits and general programme-support records are excluded from this candidate list.
- The candidate dataset includes references to instruments such as the MSMED Act, the Indian Contract Act, CERT-In directions, state startup policies and state-specific records.
- The checker reuses contract requirement extraction, semantic retrieval and structured conflict reasoning to identify clauses that may need attention.
- Source text, jurisdiction and source metadata are retained where available.

5. Complete Technology Stack

Table 1: Technologies and libraries used

Technology / library	Role in the project	Implementation note
Python	Main programming language	Application logic, data processing, NLP and tests.
Streamlit	Web application framework	Provides the interactive user interface and module tabs.
Sentence Transformers	Text embedding and semantic retrieval	Matcher defaults to all-MiniLM-L6-v2; the model is loaded lazily.
Transformers	NLP model ecosystem dependency	Required by the sentence-transformer stack; no model is fine-tuned in this project.
PyTorch	Model execution dependency	Required by the sentence-transformer / transformer stack.
NumPy	Vector operations	Used for embedding arrays and similarity matrix calculations.
Pandas	Tabular data processing	Used in the application and data workflows.
openpyxl	Excel workbook reading	Supports the state-policy workbooks.
PyMuPDF	PDF text extraction	Reads text from PDF pages. Scanned or image-only PDFs would need OCR, which is not part of the current ingestion path.

Technology / library	Role in the project	Implementation note
python-docx	DOCX text extraction	Reads paragraphs and tables from Word documents.
Pydantic	Data schema support	Listed as a dependency for core schema modelling.
pytest	Automated testing	Runs unit and integration-oriented tests.
langdetect	Language detection dependency	Listed in requirements; language detection in the shared engine currently defaults to English, so multilingual support is a future extension.
Git and GitHub	Version control and repository hosting	Tracks source changes and collaboration branches.

Dependency versions are specified in requirements.txt using minimum-version constraints; the repository does not include a fully pinned lockfile. To run the application, install the dependencies with `pip install -r requirements.txt` and start it with `streamlit run app/app.py`.

6. Datasets Used

Table 2: Datasets and data sources

Dataset / source	Contents and size	Purpose / preparation
Tamil Nadu Startup and Innovation Policy 2023	Policy source used to create the Tamil Nadu requirements.	Policy statements were converted into structured requirement rows with IDs, domains, conditions, thresholds, evidence text, page references and source URLs.
Karnataka Startup Policy 2022-2027	Policy source used to create the Karnataka requirements.	Policy statements were converted to the shared requirement schema for comparison with Tamil Nadu.
Canonical state requirements	93 structured rows across Tamil Nadu and Karnataka; stored in state_requirements_canonical.csv.	Common structured representation used by Module 1. Includes jurisdiction, requirement type, operator, threshold, unit, benefit, evidence and source metadata.
State contract regulation candidates	13 structured records; stored in state_contract_regulations.csv.	Candidate obligations, restrictions and exemptions used by Module 3. Records require legal and source validation before operational reliance.
CUAD (Contract Understanding Atticus Dataset)	Public research dataset: 510 commercial contracts, 41 clause categories and 13,000+ expert annotations.	Used as a contract-clause reference and benchmark source and mapped to internal policy topics. The project contains CUAD-related data and mapping files.
CUAD-to-policy mapping	22 mapping rows in cuad_policy_mapping.csv.	Connects CUAD clause categories to the project's internal policy topics, with mapping-strength labels.

Dataset / source	Contents and size	Purpose / preparation
Synthetic internal policy corpus	21 project-created policies, POL-01 to POL-21.	Provides a controlled comparison corpus with numbered requirements and explicit thresholds. These are synthetic examples modelled on common corporate standards, not real confidential company policies.
Company demo profiles	Five JSON profiles in data/profiles/.	Fictional profiles used to demonstrate profile-based evaluation, including positive, negative and incomplete-fact scenarios.
Evaluation and test data	Synthetic evaluation set, CSV samples, test clauses and test documents.	Used for automated tests and reported evaluation results. Not a substitute for a large, independently collected real-world test set.

6.1 Internal policy topics

The 21 synthetic policy documents cover vendor management, data privacy and protection, payment terms, confidentiality/NDA, termination notice, limitation of liability, indemnification, IP assignment, non-compete/non-solicit, insurance, audit rights, dispute resolution and governing law, force majeure, auto-renewal, assignment/change of control, exclusivity/MFN, warranty, information security, data retention, environmental compliance and third-party data sharing.

6.2 Data quality and provenance

- State-policy records preserve evidence text, source URL and page number where those details are available in the source material.
- Missing source fields are left blank rather than inferred.
- The internal policy corpus is synthetic by design because real corporate policies are generally confidential.
- CUAD does not cover every internal policy topic, so synthetic clauses are used for topics without a matching CUAD category.
- The state contract candidate file includes some records whose source descriptions are standards or guidelines rather than primary legislation. These records should be treated cautiously and reviewed against the original official instrument.

7. Prompts Used for Website Construction

The main prompts that guided the construction of the application are listed below.

Table 3: Key prompts used during development

No.	Area	Prompt
1	Interface	Build a Streamlit app with three tabs: state fit for Tamil Nadu vs Karnataka, contract vs company policies, and contract vs state regulations. Show the evidence for every result.
2	State fit	Write a profile evaluator that checks a startup's details against the state policy rows. If a fact is missing, mark it UNKNOWN instead of FAIL.
3	Contract check	Take a contract clause, find the closest policy lines using embeddings, then compare the numbers (days, caps, amounts) and flag a CONFLICT only when the values actually clash.
4	Upload and tests	Add PDF, DOCX and TXT upload support, and write pytest cases for the text extraction and the conflict checks.

8. NLP Pipeline

The project uses a hybrid pipeline. It combines NLP-based candidate retrieval with deterministic, rule-based reasoning. This distinction is important: semantic similarity identifies text that may discuss the same subject, while structured rules determine whether the terms conflict.

Table 4: Stages of the NLP pipeline

Stage	What happens	Main implementation
1. Input / ingestion	Read profile data or extract text from PDF, DOCX and TXT documents.	Streamlit UI; core/engine.py; PyMuPDF; python-docx
2. Cleaning and segmentation	Normalise document text and split it into clauses, numbered requirements, bullet points or candidate text units.	modules/upload_checker.py and shared extraction utilities
3. Requirement extraction	Identify obligation type, action, object, recipient, authorisation, quantities, unit, deadline, condition and exception.	core/extraction.py; core/quantities.py; core/schemas.py
4. Semantic representation	Encode text as vectors so that clauses can be compared by semantic similarity.	core/embeddings.py; Sentence Transformers or hashing baseline
5. Candidate retrieval	Rank policy requirements that are likely to discuss the same topic as a contract clause.	core/matching.py; cosine similarity; top-k and threshold
6. Structured comparison	Compare numeric thresholds, units, quantity types, permissions, prohibitions and authorisation conditions.	core/conflict.py; core/quantities.py
7. Verdict and explanation	Return a supported label, evidence and explanation. Use UNCERTAIN when a safe comparison cannot be made.	core/conflict.py and module workflows

Stage	What happens	Main implementation
8. Display and export	Present profile summaries, clause-level results and source details in the web interface.	app/app.py; Pandas for tabular processing and export

8.1 Module-specific pipeline

Module 1: Company profile → load state requirements → evaluate declarative profile rules → assign a verdict to each requirement → summarise confirmed benefits, potential benefits, failures, required actions and missing facts.

Module 2: Upload contract and internal policy → extract document text → split into clauses/requirements → extract structured fields → retrieve relevant policy lines using embeddings → compare the structured requirements → show conflict label, evidence and explanation.

Module 3: Select jurisdiction → load state-law candidate records → filter to relevant candidate requirement types → extract and retrieve matching contract clauses → apply the same structured comparison logic → show the candidate source and review outcome.

8.2 Conflict labels

Table 5: Conflict labels used in Modules 2 and 3

Label	Meaning
CONFLICT	The available structured evidence supports a direct mismatch between the contract clause and the policy requirement.
PARTIAL_CONFLICT	Some part of the clause conflicts, while another part may be compatible or the mismatch is limited.
NO_CONFLICT	The compared requirements are sufficiently comparable and no conflict is found under the implemented rules.
UNCERTAIN	The available information is insufficient or the quantities/conditions are not safely comparable. Human review is required.

8.3 State-fit verdicts

Table 6: Verdicts used in Module 1

Verdict	Meaning
PASS	The profile satisfies the supported condition.
FAIL	The profile does not satisfy a condition that applies to it.
APPLIES	The provision is relevant as a policy-level allocation or provision; it is not necessarily an individual award.

Verdict	Meaning
POTENTIAL	The company may qualify, but additional evidence or discretionary approval is needed.
UNKNOWN	A required profile fact is missing or cannot be evaluated.
NOT_APPLICABLE	The requirement does not apply to this profile.
INFO	The record is informational and is not a company pass/fail test.

9. Users, Inputs and Outputs

9.1 Intended users

- Startup founders and operations teams who want an initial view of state startup-policy eligibility and support.
- Compliance, procurement, legal-operations and contract-management teams who need to screen vendor or service agreements against internal policies.
- Legal reviewers or consultants who want to prioritise clauses and inspect source evidence.
- Researchers and students studying applied NLP, structured rule evaluation and contract analytics.

The platform is intended for preliminary screening and decision support. A qualified legal professional should confirm legal interpretation, applicability and enforceability.

9.2 Inputs

Each module takes its inputs through the Streamlit interface. Table 7 lists the inputs, and Figures 2 to 4 show the input screen of each module.

Table 7: Inputs accepted by each module

Module	Input	Type / format	Examples or notes
Module 1	Company name	Text	Name of the startup or demo profile.
Module 1	Industry / domain	Selection	Industry category selected from the application options.
Module 1	Legal form	Selection	Company/entity type.
Module 1	Company age	Number	Age in years.
Module 1	Annual turnover	Number (INR)	Absolute rupees; for example, ₹50 crore is represented as 500,000,000.
Module 1	Employee count	Integer	Number of employees.

Module	Input	Type / format	Examples or notes
Module 1	Additional company facts	Boolean, number, text or list values	Recognition, location, inclusion, IP, incubation and other facts used by individual policy rules. Unknown values are retained as unknown.
Module 2	Contract document	PDF, DOCX or TXT upload; alternatively demo text	The agreement to be reviewed.
Module 2	Internal policy document(s)	One or more PDF, DOCX or TXT uploads; alternatively built-in corpus	The company policies against which the contract is compared.
Module 2	Retrieval configuration	Selection / numeric	Embedding backend, similarity threshold and top-k candidate count.
Module 3	Jurisdiction	Selection	Tamil Nadu or Karnataka.
Module 3	Contract clause or document	Text area or supported document input	A clause or contract to compare against the selected jurisdiction's candidate requirements.

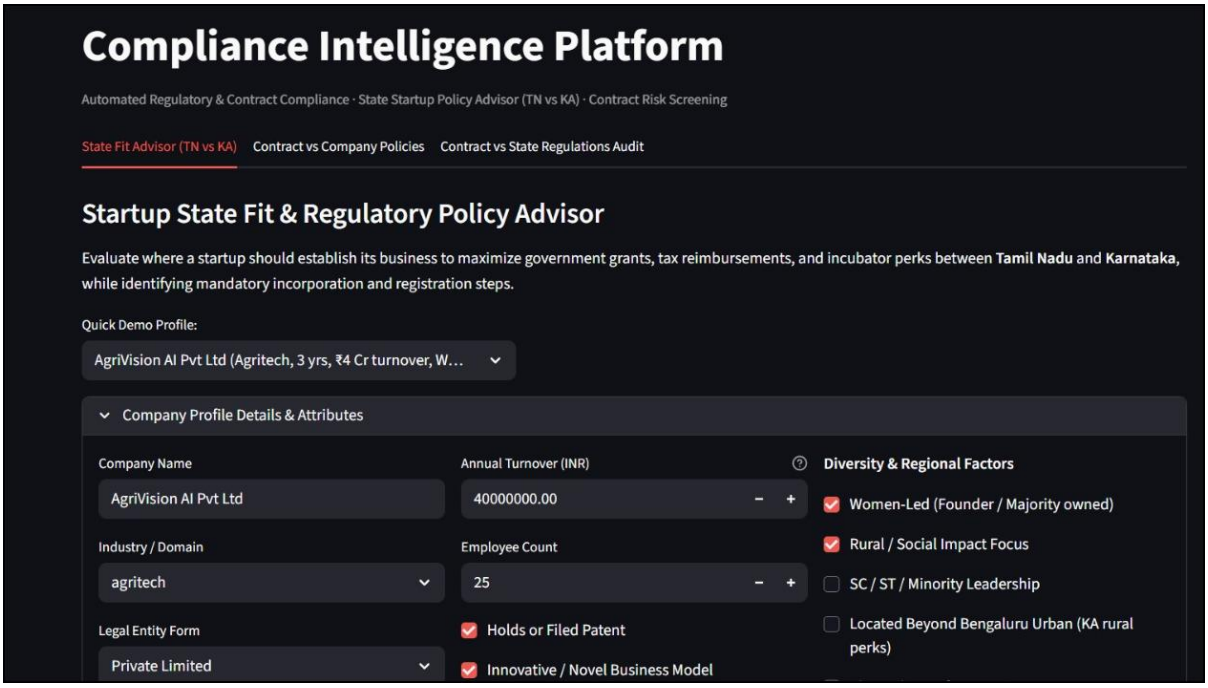


Figure 2: Module 1 input screen: company profile and diversity/regional factors

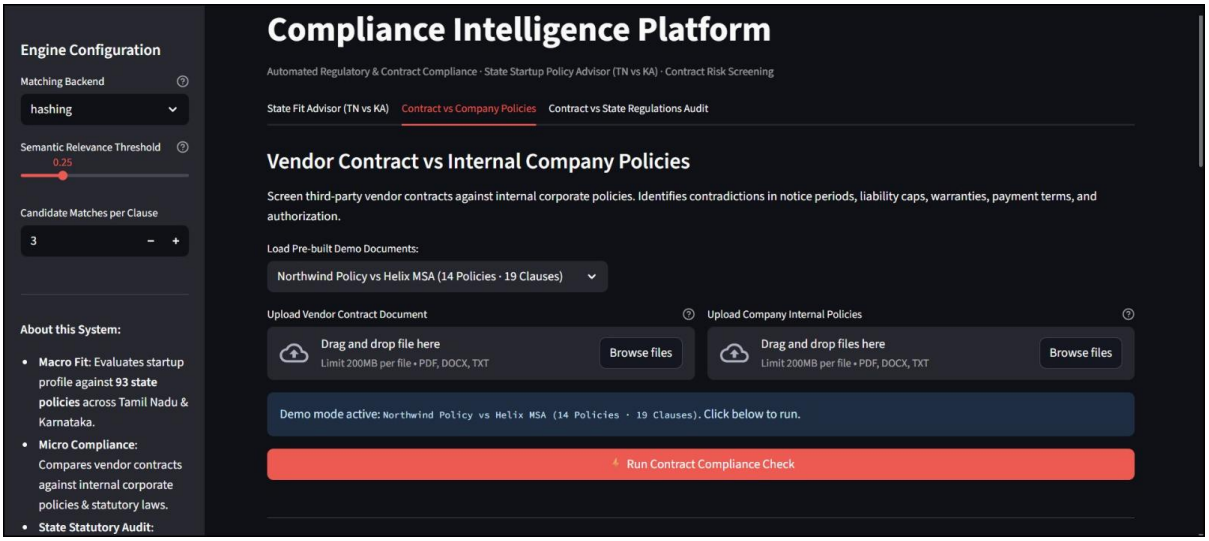


Figure 3: Module 2 input screen: contract and policy upload, demo scenario and engine settings (sidebar)

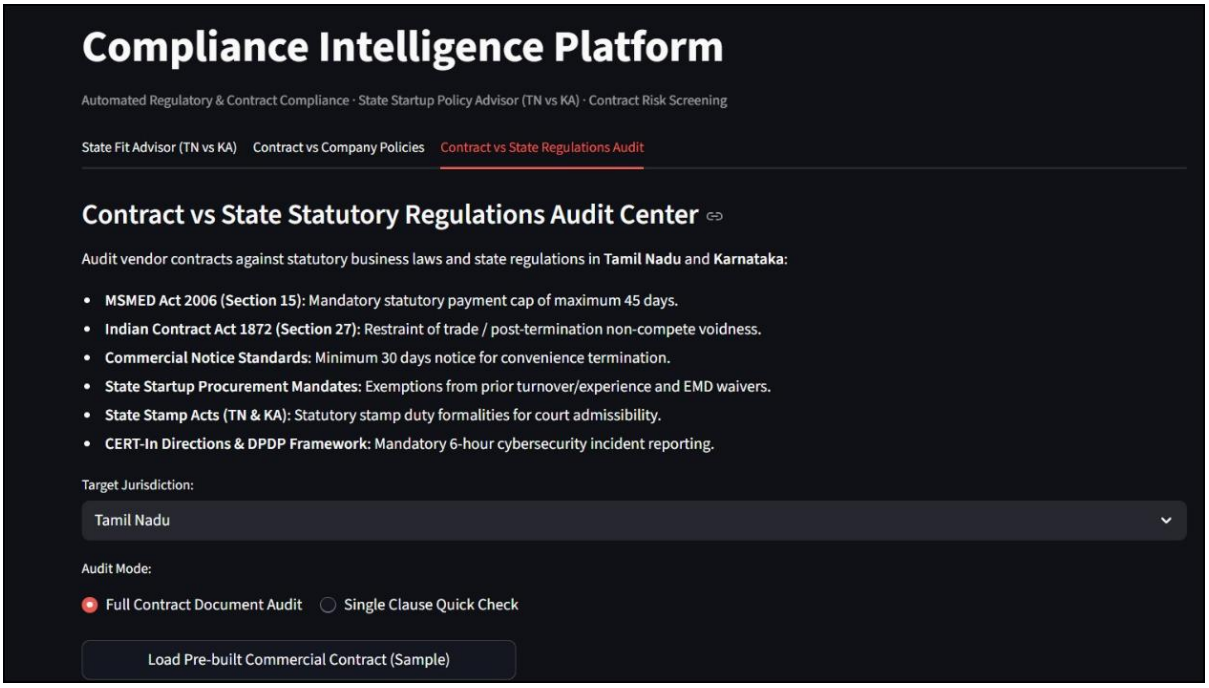


Figure 4: Module 3 input screen: target jurisdiction and audit mode

9.3 Outputs

Table 8 summarises the outputs of each module. Figures 5 to 9 show the corresponding result screens.

Table 8: Outputs produced by each module

Module	Output	Description
Module 1	Requirement-level verdicts	PASS, FAIL, APPLIES, POTENTIAL, UNKNOWN, NOT_APPLICABLE or INFO for each evaluated requirement.

Module	Output	Description
Module 1	State comparison summary	Shows eligibility failures, actions required, confirmed/potential support and missing facts; not a universal legal or financial ranking.
Module 1	Evidence and source details	Policy name, requirement ID, evidence text, page number and source URL where available.
Module 1	Export	CSV export of evaluation data is available through the interface.
Module 2	Clause-level conflict results	CONFLICT, PARTIAL_CONFLICT, NO_CONFLICT or UNCERTAIN.
Module 2	Reasoning evidence	Matched policy requirement, similarity score, comparison details, triggered rule and explanation.
Module 3	Regulatory candidate results	Relevant jurisdiction-filtered candidate records with their source text and metadata.
Module 3	Review indication	Potential contract-to-regulation mismatch or an uncertain result for human verification.

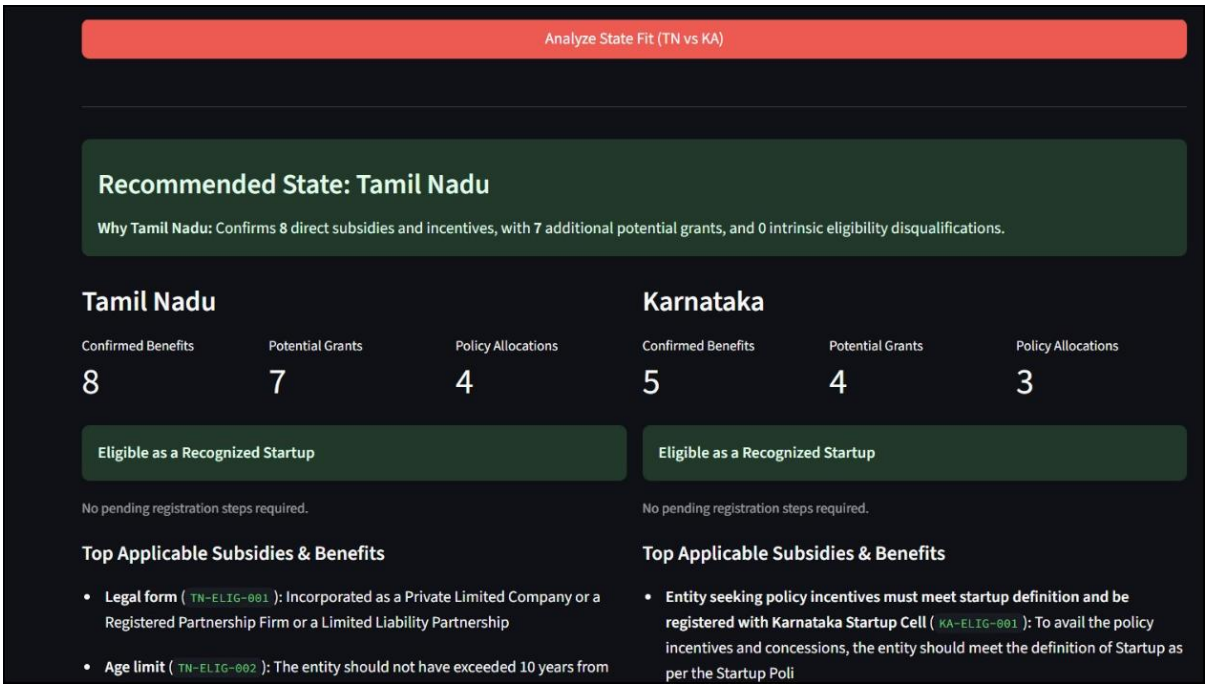


Figure 5: Module 1 output: recommended state and Tamil Nadu vs Karnataka comparison

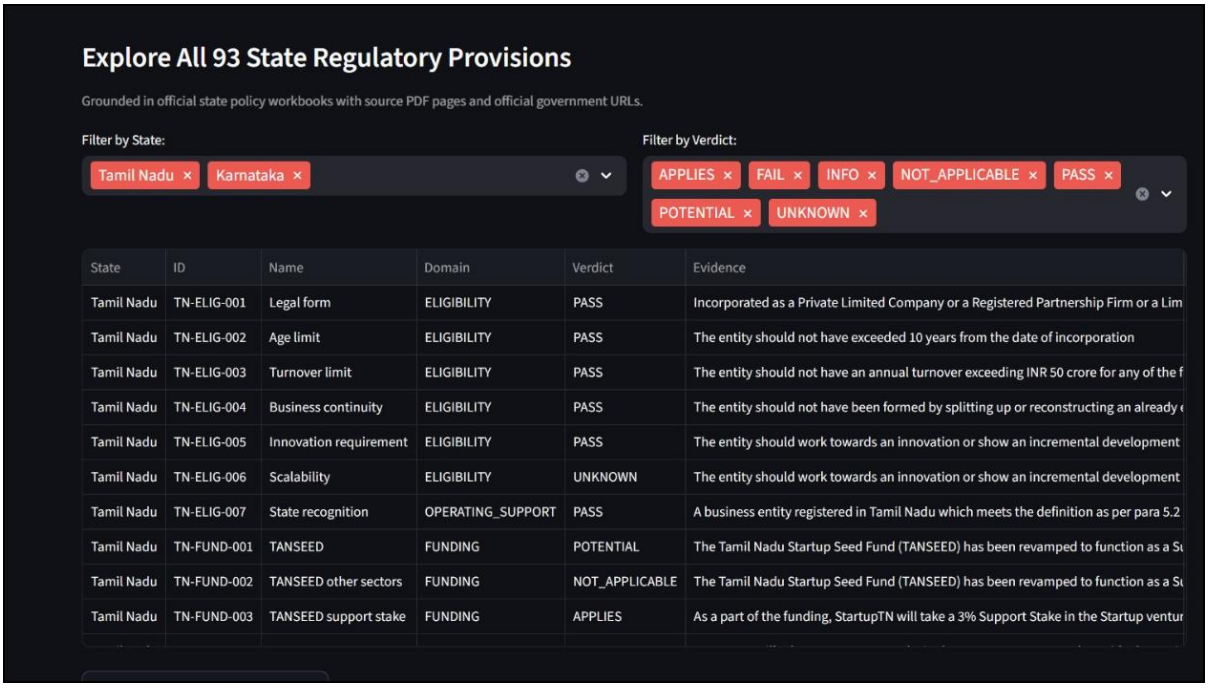


Figure 6: Module 1 output: explorer for all 93 state regulatory provisions with verdict filters

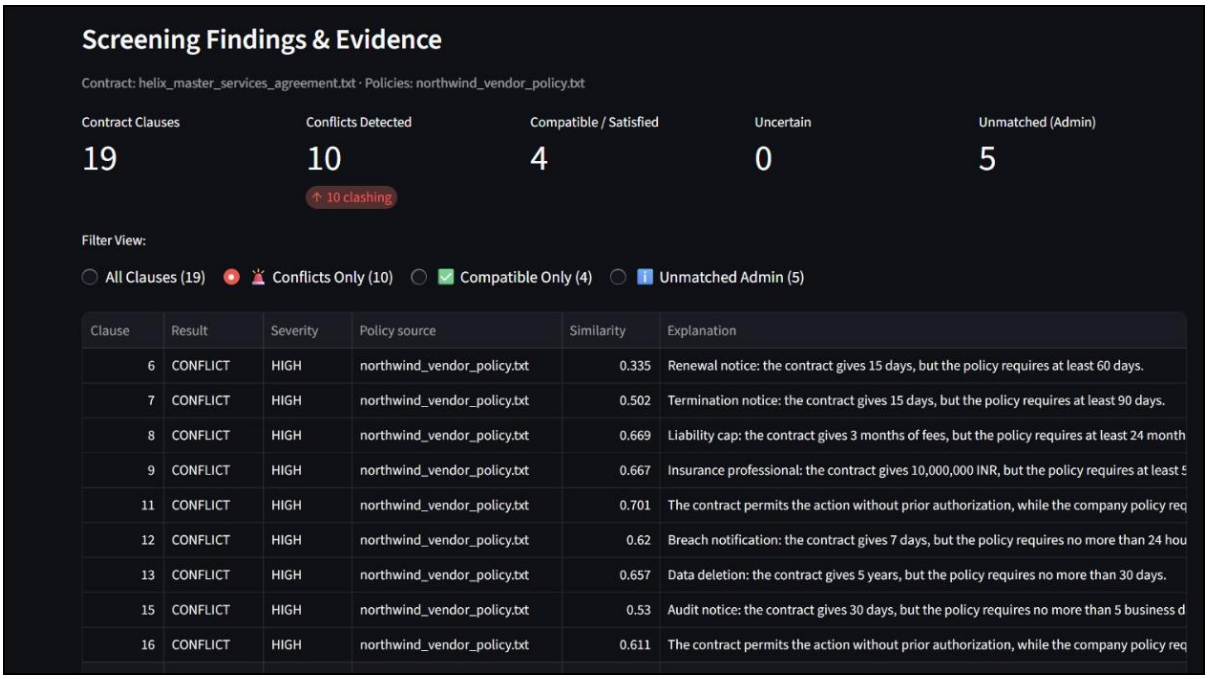


Figure 7: Module 2 output: screening findings and clause-level conflict table

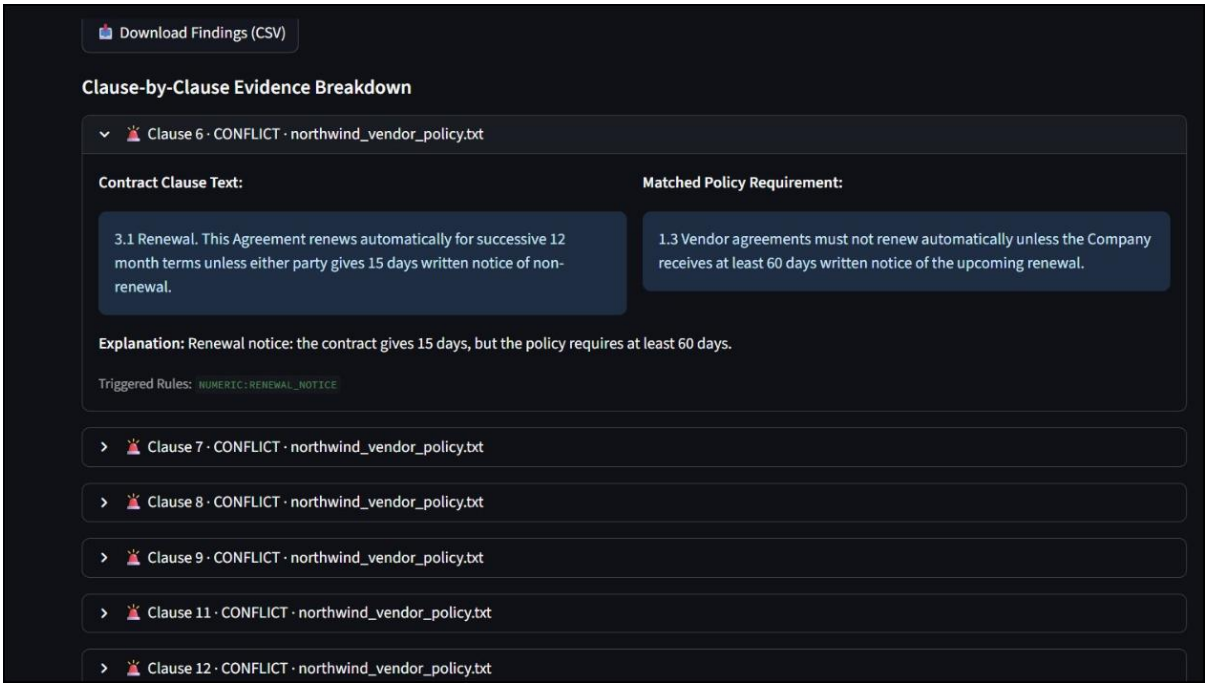


Figure 8: Module 2 output: clause-by-clause evidence breakdown with matched policy requirement

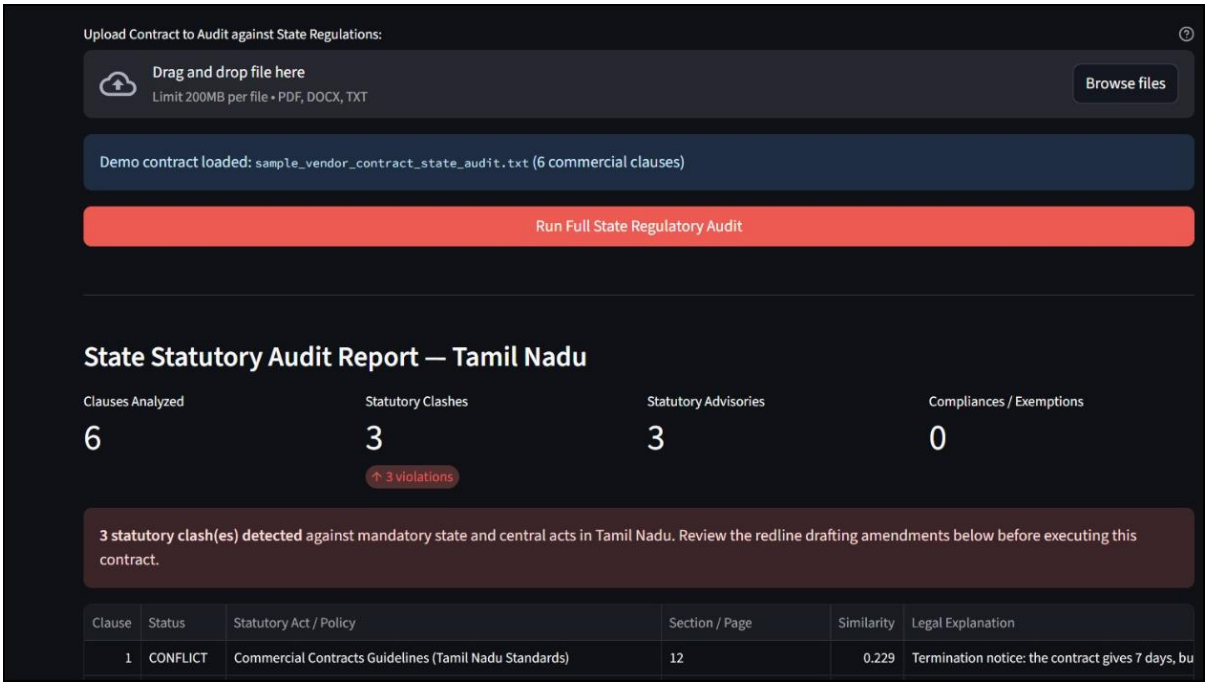


Figure 9: Module 3 output: state statutory audit report for Tamil Nadu

9.4 Example scenarios

Example 1 (State fit): A startup enters its age, turnover, employee count and industry. If a policy requires turnover below a stated ceiling and the entered turnover is above it, the rule may return FAIL. If the profile does not include a required fact, the result remains UNKNOWN.

Example 2 (Internal contract review): A company policy requires at least 30 days' termination notice, while a vendor contract allows termination on 10 days' notice. If the two requirements are comparable and the implemented numeric rule confirms the direction of the mismatch, the system can flag a conflict and show both pieces of text.

Example 3 (State regulation screening): A contract says an eligible micro or small enterprise supplier will be paid 60 days after invoice acceptance. The reviewer can compare that clause with the 45-day outer limit in Section 15 of the MSMED Act, subject to confirming that the supplier and transaction fall within the Act.

10. Evaluation, Results and References

10.1 Reported evaluation results

The evaluation report in the repository (results/eval_report.md) records 81 synthetic Module 2 test cases with 81/81 correct (100.0%) across CONFLICT, PARTIAL_CONFLICT, NO_CONFLICT and UNCERTAIN. The same report records top-5 retrieval of the gold policy line in 54/81 cases (66.7%) and a mean reciprocal rank of 0.584 for the hashing-based retrieval baseline over 93 policy lines.

The profile evaluation report (results/profile_eval_report.md) records 77/86 agreement (89.5%) with hand labels for the AgriVision AI state-fit profile: 38/40 for Tamil Nadu and 39/46 for Karnataka. The report lists nine disagreements for review.

These figures should be interpreted carefully. The 81 Module 2 cases are synthetic and small, so 100% on this test set does not establish real-world legal accuracy. Module 1 agreement is measured against project hand labels, not an independent external legal gold standard. The results are therefore best described as prototype evaluation results.

10.2 Limitations and responsible use

- The system does not provide legal advice and cannot replace review by a qualified lawyer.
- The state-policy dataset contains 93 structured records, not every current law, scheme or amendment in either state.
- The state-law dataset contains 13 candidate records. Source authority, current status, territorial scope and applicability need verification.
- The 21 internal policies and many test cases are synthetic; actual company policies may use different wording and structures.

- Scanned PDFs may not yield extractable text because OCR is not implemented in the current ingestion function.
- The shared engine includes incomplete multilingual functions: language detection currently defaults to English and translation of non-English text is not implemented.
- The Drift Monitor is a roadmap item, not a completed feature.
- Embedding similarity may miss relevant clauses or retrieve false positives; structured checks also depend on the quality of extraction and the encoded rules.

10.3 References

[1] Government of Tamil Nadu, StartupTN, Tamil Nadu Startup and Innovation Policy 2023, 2023. [Online]. Available: https://startuptn.in/pdf/Tamil-Nadu-Startup-and-Innovation-Policy-2023.pdf

[2] Government of Karnataka, Department of Electronics, Information Technology, Biotechnology and Science & Technology, Karnataka Startup Policy 2022–2027, 2022. [Online]. Available: https://static.investindia.gov.in/s3fs-public/2023-06/Startup_Policy_Karnataka.pdf

[3] Government of India, The Micro, Small and Medium Enterprises Development Act, 2006, Act No. 27 of 2006. [Online]. Available: https://www.indiacode.nic.in/bitstream/123456789/5872/1/a2006-27.pdf

[4] Government of India, The Indian Contract Act, 1872, Act No. 9 of 1872. [Online]. Available: https://www.indiacode.nic.in/bitstream/123456789/2187/2/A187209.pdf

[5] Indian Computer Emergency Response Team (CERT-In), Directions under Sub-section (6) of Section 70B of the Information Technology Act, 2000, Apr. 28, 2022. [Online]. Available: https://www.cert-in.org.in/PDF/CERT-In_Directions_70B_28.04.2022.pdf

[6] D. Hendrycks, C. Burns, A. Chen, and B. Ball, "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2021. [Online]. Available: https://arxiv.org/abs/2103.06268

[7] The Atticus Project, Contract Understanding Atticus Dataset (CUAD), 2021. [Online]. Available: https://www.atticusprojectai.org/cuad/

[8] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019. [Online]. Available: https://arxiv.org/abs/1908.10084

[9] Sentence Transformers, all-MiniLM-L6-v2: Pretrained Sentence Embedding Model. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[10] G. Filandrianos, J. S. Menezes, C. Zerva, and A. Gianola, “ARCCS: An Automated Regulatory Compliance Checking System,” in Proceedings of EMNLP 2026: System Demonstrations, 2026. [Online]. Available: https://aclanthology.org/2026.emnlp-demos.10/

[11] A. Tauqeer, A. Kurteva, T. R. Chhetri, A. Ahmeti, and A. Fensel, “Automated GDPR Contract Compliance Verification Using Knowledge Graphs,” Information, vol. 13, no. 10, Art. no. 447, 2022. [Online]. Available: https://doi.org/10.3390/info13100447

[12] M.-Y. Kim, J. Rabelo, K. Okeke, and R. Goebel, “Legal Information Retrieval and Entailment Based on BM25, Transformer and Semantic Thesaurus Methods,” The Review of Socionetwork Strategies, vol. 16, pp. 157–174, 2022. [Online]. Available: https://doi.org/10.1007/s12626-022-00103-1

[13] A. Louis, G. van Dijck, and G. Spanakis, “Finding the Law: Enhancing Statutory Article Retrieval via Graph Neural Networks,” in Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL), pp. 2761–2776, 2023. [Online]. Available: https://aclanthology.org/2023.eacl-main.203/

[14] D. Tuggener, P. von Däniken, T. Peetz, and M. Cieliebak, “LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts,” in Proceedings of the 12th Language Resources and Evaluation Conference (LREC), 2020. [Online]. Available: https://aclanthology.org/2020.lrec-1.155/

[15] J. Padhy, M. Jagannathan, and V. S. K. Delhi, “Application of Natural Language Processing to Automatically Identify Exculpatory Clauses in Construction Contracts,” Journal of Legal Affairs and Dispute Resolution in Engineering and Construction, vol. 13, no. 4, 2021. [Online]. Available: https://doi.org/10.1061/(ASCE)LA.1943-4170.0000505

[16] C. Sansone and G. Sperlí, “Legal Information Retrieval Systems: State-of-the-Art and Open Issues,” Information Systems, vol. 106, Art. no. 101967, 2022. [Online]. Available: https://doi.org/10.1016/j.is.2021.101967

[17] Sentence Transformers, “Semantic Textual Similarity.” [Online]. Available: https://sbert.net/docs/sentence_transformer/usage/semantic_textual_similarity.html